



American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Natural Language Processing

Final-Term Project Report

Spring 2025-2026

Section: A

Group: 6

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name: SHARIAR HASAN SIFAT ID: 23-50674-1	Collect & clean Bengali agri Q&A dataset, Annotate 200–500 sample QA pairs, Run baseline comparison (TF-IDF vs model)
2.	Name: ABTAHI ISLAM ID: 23-50434-1	Build FastAPI REST backend, Create simple agricultural Q&A dataset (crop diseases, fertiliser tips), Retrieve relevant answers
3.	Name: Md Toyeb Ali ID: 23-51819-2	Set up DistilBERT, Connect model to the backend API, Connect Frontend and Backend and Deployment
4.	Name: Md Mushiur Rahman ID: 23-50368-1	Build a Streamlit or simple React chat UI, English text input box + response display. Also deployment of frontend, backend and model

AgriBot - Agricultural Advisory Chatbot for Farmers

SHARIAR HASAN SIFAT, ABTAHI ISLAM, MD TOYEB ALI, MD MUSHIUR RAHMAN

American International University-Bangladesh, Dhaka, Bangladesh

23-50674-1@student.aiub.edu,

23-50434-1@student.aiub.edu, 23-51819-2@student.aiub.edu, 23-50368-1@student.aiub.edu

Abstract

Most farmers in South Asia, especially in Bangladesh and India, do not get proper advice about which crops to grow, how much fertilizer to use, or what diseases are hurting their plants. They depend on neighbors or local shops, which gives them wrong information and reduces their income. This problem is important because agriculture is the main source of living for millions of people, and bad decisions cause food shortage and financial loss in Bangladesh. The project AgriBot v4, an AI chatbot that farmers can talk to like a regular chat app. A farmer types a question in simple language, such as "What crop should I grow in Punjab?" or "Why are my wheat leaves turning yellow?", and the bot replies with a clear answer in seconds. The system uses machine learning (Random Forest, XGBoost), deep learning (DistilBERT, BERT for understanding language). AgriBot v4 is better than other solutions because it works without the internet for basic questions (all models run locally), gives advice specific to 30 Indian states using real soil and climate data as uses Indian state dataset as its primary source to deliver more accurate, region-focused recommendations and achieves 99.5% accuracy for crop recommendation and 97.6% accuracy for fertilizer advice.

1 Introduction

Agriculture is the backbone of South Asian economies. However, most small farmers lack access to expert advice. They do not know which crop is best for their soil, how much fertilizer to apply, or how to identify a plant disease early. This causes low yield, wasted money on wrong fertilizers, and crop loss due to untreated diseases.

The main problems we want to solve are:

1. Farmers do not know which crop suits their soil and weather.
2. They over-apply or under-apply fertilizers, wasting money.
3. Plant diseases are identified too late, causing large losses.
4. No single, free, simple tool exists that handles all three problems.

We solved this by building a chatbot called AgriBot v4. The farmer types a question in plain English. The system first understands what the farmer is asking (intent detection using DistilBERT), then calls the correct AI model: a Random Forest for crop recommendation, another Random Forest for fertilizer advice, a knowledge database for disease treatment, and a live weather API for planting time advice. The bot then gives back a simple, friendly answer.

The system is built with Python (FastAPI for the backend), a lightweight HTML/JavaScript frontend, and all models trained on publicly available Kaggle datasets. The entire project can run on a normal laptop without needing cloud GPU resources at inference time.

2 Background: AgriBot v4 is made of five main layers that work together. Here is the architectural overview:

Layer 1 Frontend (index.html): A simple single-page chat interface built in HTML and JavaScript. The user types a message and presses send. The page sends the message to the backend using a POST request to the /chat API endpoint and shows the reply as a chat bubble.

Layer 2 API Layer (FastAPI, api/main.py): This is the brain of the system. When a message arrives, main.py does three things: (a) it uses regex and word maps to pull out numbers like N=90, P=42 and keywords like soil type or state name from the message; (b) it fills in missing values from state_profiles.json which contains average soil and climate data for 30 Indian states; (c) it calls the DistilBERT intent classifier to decide what the farmer is asking about.

Layer 3 Intelligence Layer (predictor.py, chatbot.py, reasoning_engine.py, weather_api.py): Once the intent is known, the request is routed. reasoning_engine.py handles specific dosage or suitability questions directly from a hardcoded expert database. For general questions, predictor.py loads the correct .pkl model file and runs the prediction. chatbot.py formats the prediction result into a human-readable response. weather_api.py calls OpenWeatherMap if the farmer asks about current weather in a city.

Layer 4: Trained Models (models/ folder): All models are saved as files after training. crop_model.pkl is a Random Forest trained on 2200 rows of crop-soil-climate data. fertilizer_model.pkl is a Random Forest for fertilizer type prediction. intent_model/ is a fine-tuned DistilBERT for classifying the 4 intent types. state_profiles.json and crop_stats.json are lookup tables derived from real Indian agricultural statistics.

Layer 5 Preprocessing Layer (data_cleaning.py, feature_engineering.py, text_preprocessing.py, preprocessing_pipeline.py): This layer prepares all raw agricultural and text data before training or prediction. data_cleaning.py removes duplicates, handles missing values, fixes column names, and standardizes inconsistent entries. feature_engineering.py creates useful features such as soil ratios, nutrient balances, seasonal indicators, or encoded categorical values for machine learning models. text_preprocessing.py cleans farmer queries by lowercasing text, removing noise, tokenizing words, and normalizing spellings for better intent detection. preprocessing_pipeline.py combines all preprocessing steps into one automated workflow

3 Methodology

The project follows a standard machine learning pipeline with five steps: data collection, data preprocessing, model training, model evaluation, and deployment. Each step is described below.

Data Collection

- **Crop Recommendation Dataset:** Contains soil nutrients (N, P, K), temperature, humidity, pH, rainfall, and crop label. Used to train the crop prediction model.

- **Crop Yield Dataset:** Includes crop yield data along with soil and weather features. Used to analyze and predict crop production.
- **Fertilizer Prediction Dataset:** Contains soil type, crop type, N/P/K values, temperature, humidity, moisture, and fertilizer label. Used for fertilizer recommendation.
- **State Soil Data:** Contains soil information for different states. Helps improve prediction based on location.
- **State Weather Dataset (1997–2020):** Historical weather data like temperature and rainfall. Used to support crops and yield predictions.

Data Preprocessing

The preprocessing folder contains four files that prepare the data before training. Even though these files are currently not connected to the training scripts (a known gap in the project), they represent the intended preprocessing workflow.

data_cleaning.py - Dataset Cleaning

This file handles raw CSV data. It normalizes column names to lowercase, drops rows with missing values or fills them using mean (for numbers) or mode (for categories), removes duplicate rows, detects outliers using the IQR method and clips them to valid bounds, forces correct data types for soil columns like N, P, K and pH, and standardizes crop name variations such as unifying "soyabean" and "soybean" into one label.

feature_engineering.py - Feature Creation

This file takes the cleaned data and creates new, more useful features. Temperature values are grouped into categories (cold, cool, moderate, warm, hot). Humidity is grouped into five levels. NPK ratios are calculated (N_ratio, P_ratio, K_ratio, NP_ratio) so the model understands nutrient balance, not just raw numbers. pH is converted to acidity labels. Rainfall is grouped into five bands. All numerical features are scaled using StandardScaler. Categorical features are encoded using LabelEncoder. The final dataset grows from 8 original columns to over 25 engineered features.

text_preprocessing.py - Query Cleaning

This file handles user text, not datasets. It converts text to lowercase, fixes common spelling mistakes farmers make such as "phosphorous" to "phosphorus" and "yeild" to "yield", removes punctuation while keeping decimal numbers like pH 6.5, normalizes number ranges like "20-30 degrees" to "20 to 30 degrees", and identifies agricultural keywords in the query (crops, nutrients, fertilizers, diseases, soil types).

preprocessing_pipeline.py

This file does not contain its own logic. It imports the other three preprocessing files and connects them into one easy-to-use pipeline. A single call to `process_crop_recommendation_dataset()` runs cleaning, then feature engineering, then validation in sequence and returns the final model-ready DataFrame along with a complete report.

Model Training

The project trains six models in total. The main ones (crop and fertilizer) are trained locally and take under 5 minutes.

Crop Recommendation Model (`train_crop_model.py`)

Algorithm: Random Forest Classifier. Input features: N, P, K, temperature, humidity, pH, rainfall (7 features). Output: one of 22 crop labels. The training uses 80% of 2200 rows. The model also trains a separate yield regressor and computes state-level crop profiles. Final model is saved as `crop_model.pkl`. Accuracy: ~99.5%.

Fertilizer Recommendation Model (`train_fertilizer_model.py`)

Algorithm: Random Forest Classifier. Input: soil type, crop type, N, P, K, temperature, humidity, moisture. Output: one of 7 fertilizer types (Urea, DAP, MOP, etc.). Saved as `fertilizer_model.pkl`. Accuracy: ~97.6%.

XGBoost Crop Model (`train_xgboost_crop.py`)

Same training data and features as the Random Forest crop model but uses XGBoost algorithm. This model is only used for comparison. Results are saved to `comparison_crop_rf_vs_xgb.json`.

Intent Classifier DistilBERT (`train_intent_classifier.py`)

This is a fine-tuning task. We start from the pre-trained `distilbert-base-uncased` model (66 million parameters, trained on BookCorpus and Wikipedia). We add a new classification head on top and fine-tune the entire model on our custom 800-query agricultural dataset for 6 epochs using AdamW optimizer with learning rate $2e-5$. The output is a 4-class classifier. Saved to `models/intent_model/`.

Disease Detection EfficientNetB0 (`train_disease_efficientnet.py`)

Two-phase transfer learning from ImageNet. Phase 1: freeze the base model, train only the new classification head for 5 epochs. Phase 2: unfreeze the top 30 layers and fine-tune at a lower learning rate for 10 more epochs. Input: 224x224 plant images. Output: one of 38 disease labels. Trained on PlantVillage dataset.

Disease Detection YOLOv8 (`train_disease_yolo.py`)

Optional model that requires the PlantDoc dataset from Roboflow. Trains YOLOv8 nano for bounding-box disease detection. Without the dataset, this script saves placeholder benchmark metrics only.

Model Evaluation

All evaluations are handled by `evaluate_all.py`. This script loads the saved model files and runs a test suite of 100 realistic farmer queries, then produces two output files:

- `test_results.csv`: each query with predicted intent vs expected intent and pass/fail flag.
- `final_comparison_summary.json` : accuracy scores for all models with side-by-side comparison.

The main results are:

- Random Forest crop model: 99.5% test accuracy. XGBoost crop model: 99.1%. Random Forest wins by a small margin.
- Random Forest fertilizer model: 97.6% test accuracy.
- DistilBERT intent classifier: ~95% accuracy on the 4 intents. Full BERT: ~96% (0.5% better but 2x slower).

The evaluation confirms that Random Forest is the best choice for structured agricultural data because it is fast, interpretable, and highly accurate on small tabular datasets. DistilBERT is chosen over full BERT because it is 40% smaller and nearly accurate, making it practical to run on a normal laptop server.

4. Implementation

This section describes the programming environment, all library versions used, and the exact parameter values set in each part of the system.

4.1 Programming Environment

The project was developed and tested in the following environment:

Component	Details
Programming Language	Python 3.10+
Development IDE	Google Colab (training), VS Code (API development)
Training Hardware	Google Colab T4 GPU (for DistilBERT), CPU for RF/XGBoost

Inference Hardware	CPU-only laptop (all models run at inference without GPU)
Operating System	Ubuntu 22.04 (Colab), Windows 10/11 compatible
API Framework	FastAPI with Uvicorn ASGI server
Frontend	HTML + JavaScript
Model Storage Format	.pkl (scikit-learn), HuggingFace model/ folder .

4.2 Library Versions

Table 4.2 lists every library used in AgriBot v4 along with its minimum version and purpose.

Library	Version	Used For
scikit-learn	$\geq 1.3.0$	Random Forest, XGBoost wrapper, metrics
xgboost	$\geq 2.0.0$	XGBoost crop model
transformers (HuggingFace)	$\geq 4.30.0$	DistilBERT, BERT fine-tuning
torch (PyTorch)	$\geq 2.0.0$	DistilBERT training runtime
tensorflow / keras	$\geq 2.13.0$	EfficientNetB0 training
ultralytics	$\geq 8.0.0$	YOLOv8 disease detection
fastapi	$\geq 0.100.0$	REST API server
uvicorn	$\geq 0.23.0$	ASGI server for FastAPI
pandas	$\geq 2.0.0$	Data loading and manipulation
numpy	$\geq 1.24.0, < 2.0$	Numerical operations
joblib	$\geq 1.3.0$	Saving and loading .pkl model files
pydantic	$\geq 2.0.0$	Request/response validation in FastAPI
requests	$\geq 2.31.0$	Calling OpenWeatherMap API

matplotlib / seaborn	$\geq 3.7 / 0.13$	Plotting evaluation charts
Huggingface_hub		Deployment of Models

4.3 Model Hyperparameters

Table 4.3 shows the key hyperparameter values set in the training scripts. These values were chosen based on standard practices and small manual tuning on a validation split.

Model	Trees/Epochs	Max Depth	Min Leaf	LR	Sub/ColSample	Split
RF Crop	200	None (full)	1	—	—	80/20
RF Fertilizer	150	None (full)	2	—	—	80/20
XGBoost Crop	300	6	—	0.1	0.8 / 0.8	80/20
DistilBERT	—	—	—	2e-5	—	85/15

4.4 Detailed Parameter Notes

Random Forest (Crop Model)

- `n_estimators = 200`: 200 decision trees in the forest. More trees = more stable predictions.
- `max_depth = None`: trees grow until all leaves are pure. Works well on this small dataset.
- `min_samples_leaf = 1`: minimum 1 sample per leaf node.
- `random_state = 42`: fixed seed for reproducibility.
- `n_jobs = -1`: use all available CPU cores for parallel training.
- `test_size = 0.20`: 80% training, 20% testing (1760 train, 440 test rows).

XGBoost (Crop Model - Comparison)

- `n_estimators = 300`: 300 boosting rounds.
- `max_depth = 6`: maximum tree depth of 6 levels.

- `learning_rate = 0.1`: shrinkage factor applied to each tree's contribution.
- `subsample = 0.8`: 80% of rows sampled per tree (reduces overfitting).
- `colsample_bytree = 0.8`: 80% of features sampled per tree.
- `eval_metric = mlogloss`: multi-class log loss used during evaluation.

DistilBERT Intent Classifier

- Base model: `distilbert-base-uncased` (66M parameters, pre-trained on BookCorpus + Wikipedia).
- Epochs = 8, Batch size = 16 (training), 32 (validation).
- Optimizer: AdamW, `lr = 2e-5`, `weight_decay = 0.01`.
- Scheduler: linear warmup for first epoch, then linear decay.
- `max_length = 128` tokens (max query length).
- Gradient clipping: `max_norm = 1.0` to prevent exploding gradients.
- `test_size = 0.15`: 85% train, 15% validation.

4.5 API Configuration:

- **Server Deployment:**

The backend is deployed using FastAPI on a cloud platform, specifically Railway, instead of running locally on 127.0.0.1. The application is served using uvicorn with a production configuration:

- `uvicorn main:app --host 0.0.0.0 --port $PORT`

This allows the API to be publicly accessible via a generated domain.

- **CORS Configuration:**

Cross-Origin Resource Sharing (CORS) is enabled to allow communication between the frontend and backend hosted on different domains. Initially, all origins are allowed ("*"), ensuring compatibility with the deployed frontend on Netlify. This can later be restricted to specific domains for better security.

- **Main Endpoint:**

The primary API endpoint is:

- `POST /chat`

It accepts a JSON payload:

```
{ "message": "string" }
```

and returns a JSON response generated by the AI model.

- **Frontend Hosting:**

The frontend is deployed separately on Netlify rather than being served via FastAPI static files. Communication between frontend and backend is handled using HTTP requests (fetch) with the full backend URL.

- **Model Loading Strategy:**

Instead of loading models from local .pkl files, the system now loads models from Hugging Face Hub. Models are initialized once during server startup to improve performance and avoid repeated loading during each request.

- **Error Handling & Deployment Considerations:**

During deployment, issues such as incorrect local file paths and invalid model references were resolved by replacing local paths with proper Hugging Face repository identifiers (e.g., username/model_name). This ensures compatibility with cloud environments where local file systems differ from development setups.

- **API Testing & Documentation:**

FastAPI's built-in interactive documentation is available at:

- /docs

This interface allows real-time testing of endpoints and validation of request/response formats.

5. Result Analysis

This section presents the performance scores of all models in AgriBot v4 and compares them to baseline models. All results are from held-out test sets that were never seen during training

5.1 Overall Model Performance Summary

Task	Model	Accuracy	Training Time	Status
Crop Rec.	Random Forest	99.5%	~2 sec	✓ Chosen
Crop Rec.	XGBoost	99.1%	~8 sec	Baseline
Fertilizer	Random Forest	97.6%	~1 sec	✓ Chosen
Intent NLP	DistilBERT (fine-tuned)	95.2%	~90 min (CPU)	✓ Chosen
Intent NLP	Full BERT (fine-tuned)	95.8%	~3 hrs (GPU)	Baseline

5.2 Crop Recommendation — Random Forest vs XGBoost

Both models were trained on the same 1760-row training set (80% of the 2200-row Kaggle dataset) and tested on the same 440-row test set. Table 5.2 shows the detailed comparison.

Metric	Random Forest	XGBoost
Accuracy	99.5%	99.1%
Training Time	~2 seconds	~8 seconds
Inference Time	< 1 ms	~3 ms
Trees / Estimators	200	300
Max Depth	None (unlimited)	6
Interpretable	Yes (feature importance)	Moderate
Overfitting Risk	Low (ensemble)	Low (regularized)
Winner	✓ Random Forest	—

Random Forest achieved 99.5% accuracy vs XGBoost at 99.1%. The 0.4% gap is small but the key advantage of RF is speed: it trains in 2 seconds vs XGBoost in 8 seconds, and runs inference in under 1 millisecond per query. For a chatbot that needs instant replies, Random Forest is the clear winner. The dataset is also small (2200 rows) and tabular, which is exactly the setting where Random Forest performs best.

5.3 Intent Classification — DistilBERT vs Full BERT

Both models were fine-tuned on the same 800+ agricultural query dataset and tested on a 15% held-out validation split (~120 queries). All 4 intent classes were represented equally.

Metric	DistilBERT	Full BERT
Accuracy	95.2%	95.8%

F1 Score (macro)	0.951	0.957
Model Size	66M parameters	110M parameters
Training Time (CPU)	~90 minutes	~5+ hours
Inference Speed	~18 ms / query	~40 ms / query
Epochs	8	6
Batch Size	16 (train), 32 (val)	16 (train), 32 (val)
Learning Rate	2e-5	2e-5
Winner (production)	✓ DistilBERT	—

Full BERT scored 0.6% higher accuracy than DistilBERT (95.8% vs 95.2%). However, full BERT is 66% larger (110M vs 66M parameters) and takes 3+ hours on GPU vs 90 minutes on CPU for DistilBERT. At inference time, DistilBERT replies in ~18 ms compared to ~40 ms for BERT. For a chatbot, this difference is very noticeable. The 0.6% accuracy gain does not justify the extra cost, so DistilBERT was chosen for the final system.

5.4 Fertilizer Recommendation

The Random Forest fertilizer model achieved 97.6% accuracy on a 20% test split. The 7 fertilizer classes are: Urea, DAP, MOP, SSP, 14-35-14, 28-28, and 17-17-17. The model works by learning which combination of soil type, crop type, N/P/K levels, temperature, humidity, and moisture best predicts the correct fertilizer. No baseline comparison model was trained for fertilizer. Random Forest was the primary choice based on performance on the crop task.

5.5 Per-Intent Accuracy Breakdown

Table 5.5 shows estimated accuracy per intent class based on the 100-query evaluation test suite.

Intent Class	Accuracy	Correct / Total	Note
crop_recommendation	98%	26/27	Very high - numeric params help classification
fertilizer_advice	96%	24/25	High - NPK values give strong signal

crop_disease	93%	23/25	Good - symptom words are distinctive
weather_planting	94%	22/23	Good -weather keywords are clear

5.6 Key Observations

- Random Forest is the best model for structured agricultural tabular data - fast, accurate, and does not need GPU.
- DistilBERT is a very good choice for NLP intent classification when training data is small (< 1000 examples).
- The reasoning_engine.py intercept layer improves real-world accuracy by handling specific dosage questions before the ML model is even called - reducing model errors for these common queries.
- The weakest area is crop_disease detection via text (93%). This is expected because disease symptoms described in text are more ambiguous than numeric soil values.

6. Conclusion

AgriBot v4 was an AI-powered chatbot developed to support farmers in making better agricultural decisions, including crop selection, fertilizer usage, plant disease identification, and weather-based planning. It utilized machine learning models such as Random Forest and DistilBERT to deliver fast, reliable, and practical responses. The system featured a user-friendly web chat interface connected to a FastAPI backend, ensuring smooth and efficient interaction.

One of its key strengths was that it could run on a standard laptop, making it a low-cost and accessible solution, especially for farmers in rural areas with limited resources. This demonstrated that impactful technology does not always require expensive infrastructure—smart design and the right tools can make a real difference.

AgriBot v4 showed how AI could be used not just as a technological advancement, but as a meaningful solution to real-world problems in agriculture. Looking ahead, there were plans to expand its capabilities by adding Bengali language support, developing a mobile application, and integrating more advanced farming features—making it even more inclusive, practical, and powerful for the farming community.

7. AI and LLM

Declare which AI tools and LLM tools you have used in your project and what purpose you have used those. Please give detailed information as much as possible. Then list the prompts you have given. Maintain the following table.

SL #	Prompt	Tool with version	Purpose	Output satisfactory
01	"We want to develop a chatbot for our NLP course, named 'AgriBot — English-Language Agricultural Advisory Chatbot for Farmers'. What our Chatbot will actually do:" When a farmer describes in detail any kind of problem related to crop disease, weather-based planting advice, fertiliser dosing, then our machine will think about what can be done to solve this problem. Then our machine will provide a perfect solution to the farmer's problem." To perfectly train our machines, we have collected several datasets. The information for those datasets is given below:" Crop Recommendation using Soil Properties and Weather Prediction.csv: ‘ Crop Recommendation using Soil Properties and Weather Prediction Dataset Published: 5 September 2024 Version 1 DOI:10.17632/8v757rr4st.1 Contributor: Sltanu Alemu Description	Sonnet 4.6	Generate project idea how to solve?	Satisfied

	The dataset is comprehensive, encompassing various key factors critical to machine learning-based crop recommendation systems. The soil properties dataset includes detailed information such as specific locations identified by latitude and longitude coordinates, soil pH, soil color, surface soil composition, electrical conductivity, and a range of soil macro and micronutrients. These factors are essential in determining the suitability of different crops to various soil types. The crop type information in the dataset primarily comprises cereals, reflecting the major crops in the regions under study. The climate features dataset includes a broad spectrum of environmental conditions crucial for crop growth. This includes specific location data (latitude and longitude coordinates), as well as seasonal variations such as maximum and minimum temperatures, precipitation levels, humidity, wind speed and direction, surface pressure readings, and cloud cover assessments. These features provide insights into the climatic conditions that crops will experience throughout the growing season. The climate data has been sourced from the National Aeronautics and Space Administration's (NASA) cloud infrastructure. The soil properties and crop type data have been procured from the Ethiopian Agricultural Transformation Agency (ATA), ensuring that the dataset is grounded in reliable and locally relevant agricultural data. Additionally, crop production data has been obtained from the Ethiopian Statistics Service (ESS), providing a historical context for the crop types included in the dataset. The process of data collection was methodical. Initially, soil properties and crop type data were gathered. Following this, climate feature data			
--	--	--	--	--

was collected based on specific years and geographical coordinates (latitude and longitude), with an emphasis on anticipating the conditions for the upcoming season. This structured approach ensures that the dataset is well-rounded, providing a solid foundation for the development of a machine learning-based crop recommendation system. Source Link: https://data.mendeley.com/datasets/8v757rr4st/1 , Crop_recommendation.csv: , Crop Recommendation Dataset Maximize agricultural yield by recommending appropriate crops Last Updated: 5 years ago (Version 1) About this Dataset Precision agriculture is in trend nowadays. It helps the farmers to get informed decision about the farming strategy. Here, I present you a dataset which would allow the users to build a predictive model to recommend the most suitable crops to grow in a particular farm based on various parameters. Context This dataset was build by augmenting datasets of rainfall, climate and fertilizer data available for India. Data fields N - ratio of Nitrogen content in soil P - ratio of Phosphorous content in soil K - ratio of Potassium content in soil temperature - temperature in degree Celsius humidity - relative humidity in %			
--	--	--	--

	Source Link: https://www.kaggle.com/code/niteshhalai/crop-recommendation-dataset , Fertilizer Prediction.csv: , Fertilizer prediction Optimizing Fertilizer Selection Using Soil, Crop, and Climate Data Last Updated: a year ago (Version 1) About this Dataset The Fertilizer Prediction dataset contains detailed environmental and soil attributes essential for recommending the optimal fertilizer type. The features include temperature, humidity, soil moisture, soil type, crop type, and key soil nutrients—nitrogen, phosphorus, and potassium—while the target variable is the fertilizer name. This dataset enables data-driven insights for precision agriculture, helping farmers select the right fertilizer based on specific climatic and soil conditions, thereby boosting crop yield and promoting sustainable farming practices. Temparature Humidity Moisture Soil Type Crop Type Nitrogen Potassium Phosphorous Fertilizer Name 32			
--	---	--	--	--

	51 41 Red Ground Nuts 7 3 19 14-35-14 Source Link: https://www.kaggle.com/code/shamanthakreddymallu/fertilizer-prediction/notebook , ” I have already given you all the details of these files. Moreover, you can also get them by going to the short links provided in the file details. Now, we have given the datasets in a zip file named 'AgriBot.zip'. You need to analyze this properly. All of our datasets are already there. Now, to fulfill our objective, provide all the necessary code, starting from training the machine using Python to everything else, so that I don't have to browse through Google. If you want, you can use 'BERT, FastAPI'. Since after training our machine, we will design a website and connect it with that, also tell me how to do that. All the languages used in the project must be in English; Bengali cannot be used there. So, start the work perfectly for us."			
02	"Our model's API connection is working perfectly, and the model is working perfectly, so thank you very much. But the problem now is that while our model can answer some questions well, it cannot answer most questions. For example, it answered these questions nicely:‘ My rice leaves are turning yellow with brown spots!  CROP DISEASE	Sonnet 4.6	Some Question was not provided perfectly . For solving this.	Satisfied

	 Diagnosis: Nitrogen Deficiency / Yellow Leaf Virus Symptoms: Yellowing of leaves, stunted growth. Treatment: Apply urea or ammonium sulphate fertilizer. Check for viral vectors (aphids). Prevention: Maintain balanced NPK. Use virus-resistant varieties. Confidence: 69.3% My rice leaves are turning yellow with brown spots. What disease is this?  CROP DISEASE  Diagnosis: Nitrogen Deficiency / Yellow Leaf Virus Symptoms: Yellowing of leaves, stunted growth. Treatment: Apply urea or ammonium sulphate fertilizer. Check for viral vectors (aphids). Prevention: Maintain balanced NPK. Use virus-resistant varieties. Confidence: 69.7%  Is it good to plant maize at temperature 30°C, humidity 75%, rainfall 150mm?  CROP RECOMMENDATION  Crop Recommendations (based on your soil & climate data) 1. Coconut — confidence: 30.9% 2. Maize — confidence: 14.6%			
--	--	--	--	--

	3. Teff — confidence: 8.8% # Input used: N=50, P=30, K=30, Temp=30.0°C, Humidity=75.0%, pH=6.5, Rainfall=150.0mm Tip: These recommendations are based on soil nutrient levels and climate data. Consult a local agronomist for final decisions. Confidence: 44.9%' But it cannot answer the following questions:' Is it good to plant maize at 30°C, 75% humidity?  CROP RECOMMENDATION I'm not quite sure how to help with that. Try describing your crop, soil conditions, or symptoms, and I'll do my best to assist. Type help to see what I can do. Confidence: 48.2%  Can I grow potatoes in sandy soil?  CROP RECOMMENDATION I'm not quite sure how to help with that. Try describing your crop, soil conditions, or symptoms, and I'll do my best to assist. Type help to see what I can do. Confidence: 35.9%  Which fertilizer is best for flowering plants?  FERTILIZER ADVICE			
--	--	--	--	--

	I'm not quite sure how to help with that. Try describing your crop, soil conditions, or symptoms, and I'll do my best to assist. Type help to see what I can do. Confidence: 76.2%  "Which fertilizer is best for flowering plants?"  FERTILIZER ADVICE I'm not quite sure how to help with that. Try describing your crop, soil conditions, or symptoms, and I'll do my best to assist. Type help to see what I can do. Confidence: 75.3%  Which fertilizer is best for flowering plants?  FERTILIZER ADVICE I'm not quite sure how to help with that. Try describing your crop, soil conditions, or symptoms, and I'll do my best to assist. Type help to see what I can do. Confidence: 76.2% , What is the actual reason for this? Sir is saying to at least cover these four things: 🌾 Crop recommendations based on soil data 🔍 Fertilizer dosing advice 🌿 Crop disease diagnosis 🌤️ Weather-based planting guidance". But while our model works nicely for some questions, most of the time it does not work. A farmer will not understand humidity, rainfall, pH,			
--	--	--	--	--

	etc. He will describe in various ways in English, like 'it rained a little more or less', 'the soil was like this', 'the weather was not very good', etc. But my machine cannot identify these things. Now, to teach these things perfectly, if we need to train with more datasets, provide some more dataset sources that I can add here, and also tell me how to attach them to our coder. At the same time, if any further modifications or cleaning of the model is needed, do everything so that our output becomes more beautiful and perfect. You can see how we trained the model and connected it to the API in the file 'working.ipynb'. Another thing I see here is that you have written answers for some questions; the model answers those nicely, but for some questions, even if the question is different, the answer repeats the same. Our model cannot behave like this. The dataset we provide must be learned from the data properly, understanding what the perfect values are for specific things and what kind of effect they have in reality. The machine must learn these things by looking at our dataset. Then, when we test, for each individual question, it must answer based on what it has learned from previous learning. It won't work if the answers are predefined. Doing the work in this way will yield the original perfect result."			
3	This is my agribot, an agricultural-based AI agent project, I trained the model create a index.html file for frontend now I want to communicate frontend and backend of my project through fastAPI 24/7 without any cost and host this frontend and deploy it. What should I do guide me step by step. I don't know anything about this web server hosting and API. I am a beginner so guide me step by step	ChatGPT 5.5	How to Deploy frontend and backend	Satisfactory
4	I have oper https://agribotv4-production.up.railway.app/docs and send { "message": "hello" } the response status is 500 and internal error and my frontend chatbox console also said internal server error with status code 500 when I send messeage to the link https://agribot-v4.netlify.app/ and shows this error Server error: Internal error: Repo id must be in the form 'repo_name' or 'namespace/repo_name': '/app/api/./models/intent_model'. Use repo_type argument if needed.	ChatGPT 5.5	Fix Server Error	Satisfactory
5	Ok my context is I have developed an Agribot project which is basically agricultural based chatbot agent. Now I make a fronted using html,css backed using python FastAPI and train mutiple models located in my models folder. I locally trained those models in my laptop. Now I deploy my backend sever using railway and host my frontend using netlify and make a seamless connection between frontend and backend. I upload only my backend code to github and access railway to that github repo since my models folder is very lerge i can't upload it on git hub. Now I want to use hugging face for this purpose. Now guide me line by line how to deploy my models on hugging face. I am a beginner I didn't know anything about this process. So guide my step by step how to deploy on hugging face	ChatGPT 5.5	Deploy Model in Hugging Face	Satisfactory

6	To https://huggingface.co/Toyeb/agribot-models ! [rejected] main -> main (non-fast-forward) error: failed to push some refs to 'https://huggingface.co/Toyeb/agribot-models' hint: Updates were rejected because the tip of your current branch is behind hint: its remote counterpart. If you want to integrate the remote changes, hint: use 'git pull' before pushing again. hint: See the 'Note about fast-forwards' in 'git push --help' for details. This error is showing when I am trying to push my models in hugging face	ChatGPT 5.5	Git Push Error for Big File	Satisfactory
7	I use hugging face hub for hosting my models now guide me which file should I change to connect is with backend api. I use netlify for frontend (index.html) hosting and deploy my backend through railway and upload my models files in huggingface hub since it is very big file. Now to fetch the model prediction from hugging face write code that can fetch model from hugging face	Claude Haiku 4.5	Change local Model path to Huggingface repo	Medium
8	⚠️ Server error: Internal error: Repo id must be in the form 'repo_name' or 'namespace/repo_name': 'Toyeb/agribot-models/intent_model'. Use repo_type argument if needed. Still this error happens	Claude Haiku 4.5	Change local Model path to Huggingface repo fix error	Satisfactory

References

1. <https://www.kaggle.com/code/shamanthakreddymallu/fertilizer-prediction/notebook?select=Fertilizer+Prediction.csv>
2. <https://www.kaggle.com/datasets/gurudathg/crop-yield-prediction-using-soil-and-weather>
3. https://www.kaggle.com/datasets/anshumish/crop-yield-data-with-soil-and-weather-dataset?select=crop_yield.csv
4. https://www.kaggle.com/datasets/anshumish/crop-yield-data-with-soil-and-weather-dataset?select=state_soil_data.csv
5. https://www.kaggle.com/datasets/anshumish/crop-yield-data-with-soil-and-weather-dataset?select=state_weather_data_1997_2020.csv
6. <https://data.mendeley.com/datasets/8v757rr4st/1>

Project Code

GitHub Link: https://github.com/ToyebAli/agribot_v4

Huggingface Hub: <https://huggingface.co/Toyeb/agribot-models>